

Course Outline: File System Hierarchy

Title: Understanding File System Hierarchy

Prerequisite: Basic HTML/CSS (Week 1 content)

Target Audience: Absolute beginners (Day 4 of bootcamp) who have created HTML/CSS projects but haven't learned about file system organization

Total Lessons: 12

Estimated Total Length: ~6,850 words

Format: Conceptual (16.7% hands-on)

Course Description

You've already created several web projects—a portfolio, a Kick replica, forms with validation. But have you stopped to think about where those files actually live? How they're organized on your computer? Why your `index.html` can find your `style.css` file?

This course unveils the hidden structure beneath your projects. You'll learn how files and folders are organized in a hierarchy, how to describe file locations using paths, and how to understand project organization. This foundational knowledge is essential for working with Git (which you'll learn tomorrow), collaborating with others, and understanding how your projects are structured.

By the end of this course, you'll be able to "read the map" of any project's file structure, understand what paths mean, and recognize how files reference each other—critical skills for every developer.

Course Philosophy

- This course emphasizes:
- **Connecting to your experience:** Using the projects you've already built as concrete examples
 - **Visual learning:** Understanding structure through diagrams before commands
 - **Practical foundation:** Learning concepts you'll use immediately in Git and future projects
 - **Beginner-friendly approach:** No assumptions of prior file system knowledge
 - **Path of least resistance:** Understanding how to "read" file structures, not memorizing complex rules
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to File Organization	1	~450
01 - Understanding Your Project Files	3	~1,650
02 - Navigating with Paths	3	~1,750
03 - Project Organization Essentials	2	~1,100
04 - Mastery Check	2	~1,450

Section	Lessons	Estimated Words
05 - Your File System Foundation	1	~450
Total	12	~6,850

Section 00: Welcome to File Organization

00.0 Introduction: Where Your Files Actually Live 📖 (~450 words)

Learning Objectives:

- Understand what a file system is and why it matters for developers
- Connect file system concepts to projects you've already created
- Recognize the gap between "creating files" and "understanding file organization"

Content Outline:

- Welcome and course overview
- Quick reflection: "You've built 3+ projects—where are those files?"
- The invisible structure: How your computer organizes everything
- Why developers need to understand file organization
- Preview: What you'll learn and why it matters for your development journey

Transitions: This is your starting point—no prior file system knowledge required. You'll build on what you already know (creating HTML/CSS files) to understand the bigger picture of how those files are organized.

Key Principles:

- Every file lives somewhere specific in a structured hierarchy
- Understanding file organization is a developer fundamental
- File systems provide the foundation for version control, deployment, and collaboration
- You've been working with file systems; now you'll understand them

Examples:

- Your portfolio project from Day 1: Multiple HTML and CSS files working together
 - The Kick replica from Day 2: Images, HTML, CSS organized in folders
 - Forms project from Day 3: How your HTML file found your CSS file
-

Section 01: Understanding Your Project Files

01.0 From HTML to File System: Where Are Your Files? 📖 (~525 words)

Learning Objectives:

- Identify where your created files actually exist on the computer
- Understand the difference between "a file" and "a file's location"
- Recognize that GitHub Codespaces is a file system (in the cloud)

Content Outline:

- Your first HTML file: It exists somewhere specific
- What is a file system? (The organized structure where files live)
- Physical location vs. cloud location (your computer vs. Codespaces)
- Files have content AND location
- Why location matters: Finding files, linking files, organizing projects

Transitions: From Section 00's introduction, we now explore the fundamental question: Where do your files actually exist? This understanding sets the foundation for everything else.

Key Principles:

- A file is data stored at a specific location
- Every file has both content (what's inside) and location (where it lives)
- File systems organize files in a structured way
- Cloud environments (like Codespaces) are also file systems

Examples:

- Your `index.html` from the portfolio: It's not just code, it's stored somewhere
 - When you link `<link href="style.css">`, you're referencing a file location
 - Your browser finds files because they have predictable locations
-

01.1 Folders, Directories, and Hierarchy 📁 (~525 words)

Learning Objectives:

- Understand what folders/directories are and how they organize files
- Recognize hierarchy as a parent-child relationship
- Visualize file system structure as a tree

Content Outline:

- What is a folder? (A container that holds files and other folders)
- Folder = Directory (same thing, different terms)
- Hierarchy explained: Parents, children, siblings
- The tree structure: Root at the top, branches growing down
- Why hierarchy matters: Organization, navigation, referencing

Transitions: You know files exist in locations. Now you'll learn how those locations are organized in a hierarchy—like a family tree, but for files and folders.

Key Principles:

- Folders contain files and other folders (nested structure)
- Hierarchy creates relationships: parent folders, child items, sibling items
- The tree analogy: Root → Branches → Leaves (files are the leaves)
- Every folder can contain multiple items

Examples:

- Your portfolio project structure:

```
portfolio/  
├── index.html  
├── about.html  
└── css/  
    └── style.css
```

- Real-world analogy: File cabinets (folders) with folders inside them
 - **portfolio** is parent, **css** is child, **index.html** and **about.html** are siblings
-

01.2 Visualizing Your Portfolio Project Structure 🖥️ (~600 words)

Learning Objectives:

- Read and interpret directory tree diagrams
- Identify parent-child relationships in visual representations
- Apply hierarchy concepts to a familiar project

Content Outline:

- Introduction to directory tree diagrams
- How to read tree notation (├──, └──, indentation)
- Visual exercise: Your portfolio project structure
- Identifying relationships in the diagram
- Understanding nesting levels

Transitions: Theory becomes practical. You'll now practice reading visual representations of file structures using your actual portfolio project.

Key Principles:

- Visual diagrams make hierarchy concrete
- Indentation shows nesting depth
- Connecting lines show parent-child relationships
- Being able to "read" structure is a fundamental developer skill

Examples:

- Portfolio project diagram with 3-4 files and 1-2 folders
- Kick replica project (more complex structure with images folder)
- Simple vs. complex structures compared

Hands-On Exercise:

Practice: Reading Project Structures

You'll be shown 3 directory tree diagrams representing different project structures. For each:

1. Identify the root folder (topmost container)

2. Count how many files are in the structure
3. Identify which items are folders vs. files
4. Find parent-child relationships
5. Determine nesting depth (how many levels deep)

Example Diagram:

```
my-website/  
├── index.html  
├── contact.html  
├── images/  
│   ├── logo.png  
│   └── hero.jpg  
└── styles/  
    └── main.css
```

Questions to answer:

- What is the root folder?
- How many files total? How many folders?
- What is the parent of `logo.png`?
- What are the siblings of `contact.html`?
- How many levels deep is `main.css`?

Success Criteria:

- Correctly identify root folder in all 3 diagrams
- Accurately count files vs. folders
- Successfully trace parent-child relationships
- Understand nesting depth concept

Section 02: Navigating with Paths

02.0 What is a Path? Your File's Address 📄 (~575 words)

Learning Objectives:

- Understand what a path is and why it's essential
- Recognize paths as the "address" of a file
- Identify path separators and their role

Content Outline:

- Paths defined: The address that locates a file
- Why paths matter: Linking files, referencing resources, navigation
- Path separators: `/` (forward slash) creates the address
- Example paths from your projects
- Reading a path: Understanding each segment

Transitions: You can now visualize file structures. Next crucial skill: How to describe the location of any file using paths—the address system of file systems.

Key Principles:

- A path is a string that describes a file's location
- Paths use separators (/) to show hierarchy
- Each segment in a path represents a folder or file
- Paths are how files reference each other

Examples:

- Your CSS link: `<link href="css/style.css">` ← That's a path!
 - Image source: `` ← Another path!
 - Breaking down a path: `portfolio/css/style.css`
 - `portfolio` = folder
 - `/` = separator (means "inside of")
 - `css` = folder inside portfolio
 - `/` = separator
 - `style.css` = file inside css
-

02.1 Absolute vs Relative Paths Explained 📖 (~575 words)

Learning Objectives:

- Distinguish between absolute and relative paths
- Understand when each type is appropriate
- Recognize absolute paths start from root, relative paths start from current location

Content Outline:

- Absolute paths: The complete address from the very top (root)
- Relative paths: The address from where you currently are
- Real-world analogy: Complete address vs. directions from current location
- When to use absolute vs. relative paths
- Why web projects typically use relative paths

Transitions: Paths come in two types—like giving someone your complete address versus saying "two doors down." Each has its purpose, and understanding both is crucial.

Key Principles:

- Absolute path: Full address from root (always the same regardless of where you are)
- Relative path: Address from current location (changes based on starting point)
- Absolute paths start with `/` (from root)
- Relative paths start without `/` (from current location)

Examples:

- **Absolute path:** `/home/user/projects/portfolio/index.html`

- Complete address, works from anywhere
 - **Relative path:** `css/style.css`
 - From `index.html`, go into `css` folder, find `style.css`
 - Only works if you're starting from the right place
 - Real-world analogy:
 - Absolute: "123 Main Street, Anytown, USA" (complete address)
 - Relative: "Two doors down on the left" (from where you are now)
 - Why your HTML uses relative paths: `<link href="css/style.css">`
 - Works regardless of where the project folder is located
 - Makes projects portable (can move the whole folder)
-

02.2 Path Practice: Reading File Locations 📖 (~600 words)

Learning Objectives:

- Read and write both absolute and relative paths
- Determine the correct path type for a given scenario
- Apply path concepts to project structures

Content Outline:

- Practice identifying path types
- Writing paths from diagrams
- Understanding context: Where are you starting from?
- Common path patterns in web projects
- Troubleshooting: Why a path might not work

Transitions: You understand path types conceptually. Now practice reading, writing, and choosing the right paths for different situations.

Key Principles:

- Context matters: Where you're starting from determines the relative path
- Absolute paths are unambiguous but less portable
- Relative paths are flexible but require understanding context
- Most web projects use relative paths for resources

Examples:

- Portfolio project: How `index.html` references `css/style.css`
- Multi-page site: How `about.html` references shared CSS
- Images folder: Different pages accessing the same image

Hands-On Exercise:

Practice: Path Reading and Writing

Given 3 project directory structures, you'll practice:

Exercise 1: Identify Path Types Look at 5 different paths and label each as "absolute" or "relative":

- `/home/user/projects/website/index.html`
- `images/logo.png`
- `/var/www/html/style.css`
- `../css/main.css`
- `contact.html`

Exercise 2: Write Paths from Diagrams Given this structure:

```
project/
├── index.html
├── pages/
│   └── about.html
├── css/
│   └── style.css
└── images/
    └── logo.png
```

Write the relative path from:

1. `index.html` to `style.css`
2. `about.html` to `logo.png`
3. `index.html` to `about.html`

Exercise 3: Choose the Right Path For each scenario, decide if absolute or relative path is better:

- Linking CSS from HTML in same project
- Referencing a system file in the root directory
- Connecting pages in a multi-page website
- Loading an image from your images folder

Success Criteria:

- Correctly identify all path types
- Write accurate relative paths from different starting points
- Understand when to use each path type
- Recognize path patterns in web projects

Section 03: Project Organization Essentials

03.0 What is an Entry Point? 📖 (~550 words)

Learning Objectives:

- Understand what an entry point is and why it matters
- Recognize common entry point files in web projects
- Identify the entry point in your own projects

Content Outline:

- Entry point defined: The file that starts everything
- Why projects have entry points: The beginning of execution
- Common entry points in web development: `index.html`, `main.js`, `app.py`
- How entry points connect to the rest of your project
- Entry points and project organization

Transitions: You understand file structures and paths. Now learn about the special file that serves as your project's "front door"—the entry point.

Key Principles:

- Entry point = the starting file of a project or application
- Web projects typically use `index.html` as the entry point
- Entry points determine what loads first
- Understanding entry points helps you navigate unfamiliar projects

Examples:

- Your portfolio: `index.html` is the entry point
 - When you open the project, this is what loads
 - Other pages are accessed from here
 - Multi-file projects: Entry point connects everything
 - Why browsers look for `index.html` by default
 - Real-world analogy: Entry point is like the front door of a building
-

03.1 Project Root: The Starting Point 📁 (~550 words)

Learning Objectives:

- Understand what the project root is
- Distinguish between project root and process root
- Recognize why root context matters for paths

Content Outline:

- Project root defined: The topmost folder of your project
- Everything in the project lives inside or below the root
- Project root vs. system root (your entire computer)
- Why project root matters: It's the reference point for your project
- Process root: Where a running program thinks it is

Transitions: Entry points start execution; the project root contains everything. Understanding this boundary helps you organize and reference files correctly.

Key Principles:

- Project root = the main folder containing your entire project
- All project files exist within this root
- Paths in your project are often relative to project root
- Project root is portable (can move the whole folder)

Examples:

- Your portfolio project folder = project root
 - `portfolio/` contains everything
 - Move `portfolio/` anywhere, project still works
 - System root vs. project root:
 - System: `/` (your entire computer)
 - Project: `/portfolio/` (just this project)
 - When `index.html` references `css/style.css`:
 - It assumes both files are in the same project root
 - Path is relative to that root
 - Why this matters: Git (tomorrow) works with project roots
-

Section 04: Mastery Check

04.0 Common Path Mistakes to Avoid (Anti-patterns) 📖 (~550 words)

Learning Objectives:

- Identify common mistakes developers make with paths
- Understand why each anti-pattern is problematic
- Apply correct practices instead of anti-patterns

Content Outline:

- Anti-pattern 1: Hardcoding absolute paths in web projects
- Anti-pattern 2: Wrong separators (backslashes on Windows)
- Anti-pattern 3: Not understanding current location context
- Anti-pattern 4: Mixing absolute and relative paths inconsistently
- Why these mistakes happen and how to avoid them

Transitions: You've learned the correct way to work with file systems and paths. Now recognize common mistakes so you can avoid them in your own projects.

Key Principles:

- Absolute paths in web projects make projects non-portable
- Path separators must be consistent (`/` not `\`)
- Context awareness prevents "file not found" errors
- Consistency in path usage improves project maintainability

Examples:**Anti-pattern 1: Hardcoded Absolute Paths**

```
<!-- ❌ Bad: Won't work on anyone else's computer -->
<link href="/Users/yourname/Desktop/project/css/style.css">
```

```
<!-- ☒ Good: Works anywhere -->
<link href="css/style.css">
```

Anti-pattern 2: Wrong Separators

```
<!-- ☒ Bad: Windows backslashes don't work on web -->


<!-- ☒ Good: Forward slashes work everywhere -->

```

Anti-pattern 3: Lost in Context

```
project/
├── index.html
└── pages/
    └── about.html

<!-- From about.html trying to reference CSS -->
<!-- ☒ Bad: Doesn't account for being in pages/ folder -->
<link href="css/style.css">

<!-- ☒ Good: Goes up one level first -->
<link href="../../css/style.css">
```

Anti-pattern 4: Inconsistent Mixing

```
<!-- ☒ Bad: Mixing path types confusingly -->
<link href="/home/user/css/style.css">


<!-- ☒ Good: Consistent relative paths -->
<link href="css/style.css">

```

Why These Happen:

- Copying code without understanding context
- Not testing on different computers
- Confusion about starting location
- Not understanding absolute vs. relative differences

Consequences:

- Files not found (broken links, missing images)
- Projects that only work on your computer

- Difficult collaboration with teammates
- Deployment problems

How to Avoid:

- Use relative paths in web projects
 - Always use forward slashes (/)
 - Know your current location before writing paths
 - Test your projects in different locations
 - Be consistent with path strategy
-

04.1 Assessment: File System Fundamentals 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of file system hierarchy
- Apply path concepts to realistic scenarios
- Evaluate file organization decisions

Assessment Format: Multiple choice questions covering the full course

Questions:

Question 1: File System Structure Look at this directory structure:

```
website/  
├── index.html  
├── pages/  
│   ├── about.html  
│   └── contact.html  
└── assets/  
    ├── css/  
    │   └── style.css  
    └── images/  
        └── logo.png
```

What is the parent folder of `style.css`?

- A) `website`
- B) `assets`
- C) `css`
- D) `pages`

Correct Answer: C

Explanation: The `css` folder directly contains `style.css`, making it the parent. While `assets` and `website` are ancestors, they are not the immediate parent.

Question 2: Path Types Which of the following is an absolute path?

- A) `css/style.css`
- B) `../images/logo.png`
- C) `/home/user/projects/website/index.html`
- D) `pages/contact.html`

Correct Answer: C

Explanation: Absolute paths start from the root (`/`) and provide the complete address. The others are relative paths that depend on current location.

Question 3: Relative Path Writing Using the structure from Question 1, what is the correct relative path from `about.html` to `logo.png`?

- A) `assets/images/logo.png`
- B) `../assets/images/logo.png`
- C) `/assets/images/logo.png`
- D) `images/logo.png`

Correct Answer: B

Explanation: From `about.html` (inside `pages/`), you must go up one level (`..`) to reach `website/`, then into `assets/images/` to find `logo.png`.

Question 4: Entry Points In a typical web project, which file usually serves as the entry point?

- A) `style.css`
- B) `index.html`
- C) `logo.png`
- D) `about.html`

Correct Answer: B

Explanation: `index.html` is the conventional entry point for web projects. Browsers look for this file by default when opening a website directory.

Question 5: Project Root What is the project root in the structure shown in Question 1?

- A) The `css` folder
- B) The `website` folder
- C) The `assets` folder
- D) The system root `/`

Correct Answer: B

Explanation: The `website` folder is the topmost folder of the project, containing all project files and folders. This is the project root.

Question 6: Path Anti-patterns Which path usage represents an anti-pattern in web development?

- A) `<link href="css/style.css">`
- B) ``

C) `<link href="/Users/yourname/Desktop/project/css/style.css">`

D) `<a href="pages/about.html">`

Correct Answer: C

Explanation: Hardcoded absolute paths (especially with user-specific directories like `/Users/yourname/`) make projects non-portable and won't work on other computers.

Question 7: Hierarchy Understanding In a file system hierarchy, folders can contain:

A) Only files

B) Only other folders

C) Both files and other folders

D) Neither files nor folders

Correct Answer: C

Explanation: Folders (directories) can contain both files and other folders, creating the nested hierarchy structure of file systems.

Evaluation Criteria:

Score Interpretation:

- **7/7 correct:** Excellent! You have a strong understanding of file system fundamentals
- **5-6 correct:** Good! Review the questions you missed to strengthen your understanding
- **3-4 correct:** Fair. Review the course materials, especially sections on paths and hierarchy
- **0-2 correct:** Needs work. Revisit the course from the beginning, focusing on visual exercises

Key Concepts Assessed:

- Reading directory structures
- Identifying path types (absolute vs. relative)
- Writing correct relative paths
- Understanding entry points and project roots
- Recognizing anti-patterns
- Applying hierarchy concepts

Next Steps Based on Score:

- **High scorers:** You're ready to apply these concepts with command line tools
 - **Medium scorers:** Practice reading more directory structures and writing paths
 - **Low scorers:** Spend more time with visual diagrams and basic path examples
-

Section 05: Your File System Foundation

05.0 Mastering File Organization (~450 words)

Content Outline:

What You've Accomplished:

You started this course with projects you created but didn't fully understand where those files lived or how they were organized. Now you can:

- Visualize file system hierarchies and read directory structures
- Understand that files exist at specific locations in an organized tree
- Recognize parent-child relationships in file organization
- Read and write both absolute and relative paths
- Identify entry points and project roots
- Avoid common path mistakes that break projects

The Bigger Picture:

File system knowledge isn't abstract theory—it's the foundation for everything you'll do as a developer:

- **Version control:** Git (which you'll learn next) tracks changes to files in your project hierarchy
- **Collaboration:** Sharing projects means others can navigate your file structure
- **Deployment:** Uploading projects to servers requires understanding file organization
- **Debugging:** "File not found" errors make sense when you understand paths
- **Professional work:** Every project has a structure; now you can read any of them

From Understanding to Action:

You've learned to "read the map"—understanding how files and folders are organized, how to describe locations with paths, and how projects are structured around entry points and roots.

The next step is learning to navigate that map actively using command line tools. You'll learn commands to move through directories, create files and folders, and manipulate your project structure—but that foundation starts with the understanding you've built here.

Key Takeaways:

1. **Files live in hierarchies:** Everything is organized in a tree structure with parents and children
2. **Paths are addresses:** They tell you (and your computer) exactly where a file lives
3. **Context matters:** Relative paths depend on where you start; absolute paths are complete addresses
4. **Projects have structure:** Entry points and roots define project boundaries
5. **Organization enables collaboration:** Well-structured projects are easier to share, maintain, and scale

Resources for Continued Learning:

- Practice reading directory structures in any project you encounter
- Examine how professional projects organize their files (open source repositories)
- Pay attention to paths in your HTML, CSS, and future JavaScript files
- Notice how different developers structure similar projects

Your Developer Journey:

Understanding file systems is a milestone in your development journey. This knowledge separates "writing code" from "organizing projects." You're not just creating isolated files anymore—you're building structured, professional projects.

Every HTML file you create, every CSS link you write, every image you reference—you now understand the hidden system that makes it all work. That's the mark of a developer who understands their tools, not just uses them.

Congratulations on building this essential foundation. The file system is no longer invisible to you.

End of Course Outline